Resumen Completo: SQL Avanzado

Temario

- Sentencias de Manipulación de Datos (DML): Subconsultas, Operadores de Conjunto.
- Sentencias de Definición de Datos (DDL): Vistas, Índices, otros.
- Sentencias de Control de Datos (DCL).
- Sentencias de Control de Transacciones (TCL).

---

1. Subconsultas (DML)

Una **subconsulta** es una sentencia `SELECT` dentro de otra sentencia SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`). Pueden anidarse y permiten simplificar consultas complejas.

Tipos según el resultado:

- Escalar (devuelve un solo valor)
- De una sola fila
- De múltiples filas

Ubicación común:

- `WHERE`: filtra registros según el resultado.
- `HAVING`: filtra grupos.
- `FROM`: genera una tabla temporal.
- `SELECT`: muestra valores calculados.

Operadores principales:

- **Comparación:** `=`, `<>`, `>`, `<`, `>=`, `<=`
- **IN / NOT IN:** Verifica si un valor está (o no) dentro de los resultados de una subconsulta.
- **ANY / ALL:** Compara con algún o con todos los valores.
- **EXISTS / NOT EXISTS:** Evalúan si existen (o no) filas devueltas por la subconsulta.

Ejemplos:

```
-- Comparación
SELECT * FROM producto
WHERE precio >= (SELECT MAX(precio) FROM producto WHERE fabricante = 'Samsung');

-- IN
SELECT ciudadID FROM ciudad
WHERE ciudadID IN (SELECT ciudadID FROM clientes);

-- ANY
SELECT empleadoID, nombre
FROM empleados
```

```
WHERE sueldo > ANY (SELECT sueldo FROM empleados WHERE categoria = 'supervisor');

-- EXISTS
SELECT nombre FROM cliente C
WHERE EXISTS (SELECT pedidoID FROM pedido P WHERE C.clienteID = P.clienteID);
```

Correlacionadas vs No Correlacionadas

- **No correlacionada:** Independiente, se ejecuta primero.
- **Correlacionada:** Depende de la consulta externa, se ejecuta por cada fila.

En otras sentencias:

- **INSERT:**

```
INSERT INTO aprobadosMatem (alumnoID, nombre, nota)
SELECT a.alumnoID, a.nombre, c.nota
FROM alumnos a INNER JOIN calificaciones c ON a.alumnoID = c.alumnoID
WHERE c.nota > 6 AND c.materia = 'Matem';
```

- **UPDATE:**

```
UPDATE empleados
SET sueldo = sueldo * 1.1
WHERE departamentoID IN (SELECT departamentoID FROM departamentos WHERE
presupuesto > 80000);
```

- **DELETE:**

```
DELETE FROM empleados
WHERE sueldo < (SELECT AVG(sueldo) FROM empleados);
```

---

2. Operadores de Conjunto (DML)

Permiten combinar resultados de varias consultas `SELECT`:

Operador	Descripción
UNION	Une los resultados, eliminando duplicados.
INTERSECT	Devuelve las filas comunes entre dos consultas.
EXCEPT	Devuelve las filas que están en la primera consulta y no en la segunda.

> **Nota:** se puede usar `ALL` para mantener duplicados.

```
SELECT ciudad FROM clientes
UNION
SELECT ciudad FROM vendedores;
```

---

3. Vistas (DDL)

Una **vista** es una tabla virtual definida por una consulta SQL.

```
CREATE VIEW nombre_vista AS
SELECT columnas FROM tabla WHERE condición;
```

Ventajas:

- Ocultan datos sensibles.
- Simplifican consultas complejas.
- Mantienen independencia lógica.

Ejemplo:

```
CREATE VIEW v_emp_jef (empID, empNombre, jefID, jefNombre) AS
SELECT e.empID, e.nombre, j.empID, j.nombre
FROM empleados e LEFT JOIN empleados j ON e.jefeID = j.empID;
```

---

4. Índices (DDL)

Estructuras que mejoran el rendimiento de búsqueda.

```
CREATE INDEX idx_nombre ON empleados (apellido, nombre);
CREATE UNIQUE INDEX idx_correo ON empleados (email);
```

Tipos:

- **Clustered (agrupado):** define el orden físico de los datos (uno por tabla).
- **Non-clustered (no agrupado):** estructura independiente de punteros.
- **Único:** evita duplicados.
- **Compuesto:** combina varias columnas.

Buenas prácticas:

- Indexar columnas usadas en `WHERE`, `JOIN`, `ORDER BY`.
- Evitar índices en tablas pequeñas o con poca variabilidad.

---

5. Procedimientos y Funciones (DDL)

- **Procedimientos almacenados:** bloques de código SQL precompilados que pueden aceptar parámetros y modificar datos.
  Se ejecutan con `CALL` o `EXEC`.
- **Funciones:** devuelven un valor, solo parámetros de entrada, pueden usarse dentro de consultas.

---

6. Sentencias de Control de Datos (DCL)

Controlan los permisos y seguridad.

```
GRANT SELECT, INSERT ON empleados TO usuarioA;
REVOKE SELECT ON empleados FROM usuarioA;
```

- **GRANT:** otorga privilegios.
- **REVOKE:** los retira.

---

7. Sentencias de Control de Transacciones (TCL)

Administran transacciones SQL (unidad de trabajo).
Basadas en los principios **ACID**:

Principio	Descripción
A - Atomicidad	Se ejecuta todo o nada.
C - Consistencia	Mantiene estados válidos.
I - Aislamiento	Evita interferencia entre transacciones.
D - Durabilidad	Los cambios confirmados son permanentes.

Comandos TCL:

```
BEGIN TRANSACTION; -- inicia
UPDATE cuentas SET balance = balance - 100 WHERE cuentaID = 1;
UPDATE cuentas SET balance = balance + 100 WHERE cuentaID = 2;
COMMIT; -- confirma cambios
ROLLBACK; -- revierte
SAVEPOINT punto1; -- punto intermedio
```

Fin del Resumen de SQL Avanzado

🛡️ Administración de Bases de Datos – Enfoque en Ciberseguridad

Licenciatura en Ciberdefensa
Año: 2025

---

1. Seguridad en Bases de Datos

La **seguridad** en bases de datos protege la información frente a accesos no autorizados, pérdida o manipulación. En ciberseguridad, esto asegura la **confidencialidad**, **integridad** y **disponibilidad** (CIA triad).

1.1. Autenticación

Proceso de verificar la identidad de usuarios o sistemas.

Métodos comunes:

- 🔑 Usuario y contraseña.
- 🔒 Autenticación multifactor (MFA).
- 🌐 Autenticación externa (LDAP, Active Directory, SSO con OAuth/Kerberos).

Ejemplo (PostgreSQL):

```
CREATE USER analista WITH PASSWORD 'ClaveFuerte123!';
ALTER USER analista WITH ENCRYPTED PASSWORD;
```

Buenas prácticas en ciberseguridad:

- Usar contraseñas complejas (mín. 12 caracteres).
- Habilitar **SSL/TLS** para conexiones cifradas.
- Limitar intentos fallidos para evitar ataques de fuerza bruta.

1.2. Autorización

Controla qué acciones puede realizar cada usuario o rol.

```
GRANT SELECT, INSERT ON seguridad.log_eventos TO 'auditor'@'localhost';
REVOKE DELETE ON seguridad.log_eventos FROM 'auditor'@'localhost';
```

Principio de mínimo privilegio: solo otorgar lo estrictamente necesario.

1.3. Control de Acceso Basado en Roles (RBAC)

Permite administrar permisos por **roles**, no por usuarios individuales.

```
CREATE ROLE analista_ciber;
GRANT SELECT ON logs.alertas TO analista_ciber;
ALTER ROLE analista_ciber ADD MEMBER usuario1;
```

Ejemplo práctico:

- Rol `analista_ciber`: puede leer alertas.
- Rol `admin_dba`: acceso total a las tablas de incidentes.

1.4. Cifrado de Datos

El cifrado protege los datos en **reposo** y en **tránsito**.

En reposo (almacenados)

Ejemplo en Oracle:

```
ALTER TABLE incidentes ADD (descripcion VARCHAR2(255) ENCRYPT);
```

En tránsito (conexiones)

- Usar **TLS/SSL**.
- Evitar versiones obsoletas (TLS 1.0/1.1).

Escenario: Un atacante intercepta tráfico SQL → con TLS, los datos se transmiten cifrados y son ilegibles.

1.5. Auditoría de Bases de Datos

Registra eventos (quién, cuándo, qué acción realizó).

```
-- PostgreSQL con pgAudit
SET pgaudit.log = 'read, write';
```

En ciberseguridad: permite detectar accesos sospechosos o modificaciones indebidas.

Buenas prácticas:

- Guardar logs en servidor independiente.
- Configurar alertas por actividad inusual.
- Cumplir normativas (ISO 27001, GDPR).

1.6. Copias de Seguridad y Recuperación

Esencial ante ataques ransomware o errores humanos.

Tipos de backup:

Tipo	Descripción
Completo	Copia total de la base
Incremental	Solo cambios desde el último backup
Diferencial	Cambios desde el último completo

Ejemplo: Guardar copias cifradas en nube privada o almacenamiento seguro.

---

2. Concurrencia y Recuperación

Garantiza integridad cuando múltiples usuarios acceden a la base.

2.1. Propiedades ACID

- **Atomicidad:** todo o nada.
- **Consistencia:** mantiene reglas del sistema.
- **Aislamiento:** transacciones no interfieren.
- **Durabilidad:** cambios persisten ante fallos.

```
BEGIN;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

En ciberseguridad: asegura que los registros de incidentes no se pierdan ni corrompan.

2.2. Problemas de Concurrencia

Problema	Descripción	Ejemplo
Lecturas sucias	Leer datos no confirmados	Leer registro de ataque aún no validado
Lecturas no repetibles	Valor cambia entre lecturas	Contador de alertas modificado
Lecturas fantasma	Filas nuevas aparecen/desaparecen	Nuevas alertas insertadas durante una consulta

Solución: usar niveles de aislamiento (`SERIALIZABLE`, `READ COMMITTED`).

---

3. Desempeño (Rendimiento)

Optimizar rendimiento evita cuellos de botella y mejora respuesta ante incidentes.

3.1. Estrategias de Indexación

Los índices aceleran consultas, aunque pueden ralentizar escrituras.

```
CREATE INDEX idx_evento_fecha ON incidentes (fecha);
```

Ejemplo: buscar ataques por fecha o dirección IP.

Tipos útiles en ciberseguridad:

- 🔍 *Full-Text Index*: para buscar palabras clave ("malware", "phishing").
- 🗺️ *Índices espaciales*: para mapear IPs o ubicaciones.

3.2. Optimización de Consultas

Usar **EXPLAIN ANALYZE** para ver el plan de ejecución:

```
EXPLAIN ANALYZE SELECT * FROM logs WHERE tipo = 'intrusion';
```

Buenas prácticas:

- Evitar `SELECT *`
- Usar filtros (`WHERE tipo = 'malware'`)
- Indexar columnas en `JOIN`

3.3. Particionamiento y Sharding

Divide grandes volúmenes de datos (como logs de incidentes).

```
CREATE TABLE logs_2025 PARTITION OF logs FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Ventaja: búsqueda más rápida y mantenimiento por año o región.

---

4. Modelos de Procesamiento

Define cómo se organizan y acceden los datos.

4.1. Modelo Cliente-Servidor

Aplicación de monitoreo (cliente) consulta servidor de base (MySQL, PostgreSQL).

Riesgo: ataques DoS si hay demasiadas conexiones.
Mitigación: balanceo de carga y límites de conexión.

4.2. Distribución y Replicación

Replicar datos en varios nodos para disponibilidad.

Ejemplo:

- Nodo A (maestro): recibe alertas nuevas.
- Nodo B (esclavo): replica para análisis forense.

Ciberseguridad: garantiza que la evidencia se preserve incluso si un nodo falla.

4.3. Two-Phase Commit (2PC)

Asegura consistencia entre nodos distribuidos.

Ejemplo: sincronización de logs entre centros de datos.

1. Coordinador pide confirmación.
2. Si todos responden, ejecuta COMMIT; si uno falla, ROLLBACK.

4.4. Propagación Asincrónica

Actualizaciones no inmediatas → mejora rendimiento.

Ejemplo: el servidor de respaldo recibe las alertas de seguridad unos segundos después.

📓 Formatos de Intercambio de Datos y Logs

📓 Introducción

Los **formatos de intercambio de datos** permiten estructurar, almacenar y compartir información entre aplicaciones, sistemas y servicios.
Su correcta elección impacta directamente en la **eficiencia, interoperabilidad y trazabilidad** de los sistemas modernos.

---

1. 📦 Concepto de Formatos de Intercambio de Datos

Los formatos de intercambio de datos **definen la estructura** que usa un sistema para **representar información** de manera comprensible tanto por humanos como por máquinas.
Permiten la comunicación entre sistemas heterogéneos (por ejemplo, entre un servidor y un cliente web).

◇ Características ideales

- **Estandarizados** (compatibles entre plataformas).
- **Livianos y legibles.**
- **Soportados por múltiples lenguajes.**
- **Autodescriptivos:** contienen metadatos que describen los datos.
- **Extensibles y seguros.**

---

2. 🧱 Formatos Estructurados más Utilizados

◇ 1. CSV (Comma-Separated Values)

Formato tabular simple, donde los valores se separan por comas o punto y coma.

Ejemplo:

```
nombre,apellido,edad
Juan,Pérez,30
Ana,López,25
```

Ventajas:

- Ligero y fácil de generar.
- Ideal para datos tabulares.
- Compatible con Excel y sistemas ETL.

Desventajas:

- No soporta jerarquías ni tipos de datos complejos.
- Ambigüedades con caracteres especiales (comas, saltos de línea).

◇ **2. JSON (JavaScript Object Notation)** Formato ligero y flexible basado en texto. Usa pares clave: valor.

Ejemplo:

```
Copiar código
{
  "nombre": "Juan",
  "edad": 30,
  "habilidades": ["Python", "SQL"]
}
```

Ventajas:

- Legible por humanos.
- Eficiente para APIs y web services.

- Nativo en lenguajes modernos.
- Soporta estructuras anidadas.

Desventajas:

- No soporta comentarios.
- Tamaño mayor que formatos binarios.

Usos comunes: APIs REST, configuraciones, logs de aplicaciones, almacenamiento NoSQL.

◇ **3. XML (eXtensible Markup Language)** Formato de texto jerárquico y extensible.

Ejemplo:

```
Copiar código
<persona>
  <nombre>Juan</nombre>
  <edad>30</edad>
</persona>
```

Ventajas:

- Muy estructurado.
- Soporta metadatos y validación con DTD/XSD.
- Ampliamente usado en integraciones empresariales.

Desventajas:

- Verboso (mucho texto).
- Más pesado que JSON.
- Lectura y parseo más lentos.

Usos comunes: Servicios SOAP, configuración, intercambio empresarial (ERP, banca).

◇ **4. YAML (YAML Ain't Markup Language)** Formato legible para humanos, basado en indentación.

Ejemplo:

```
Copiar código
nombre: Juan
edad: 30
habilidades:
  - Python
  - SQL
```

Ventajas:

- Muy legible.
- Ideal para configuración (Docker, Kubernetes, Ansible).
- Soporta comentarios.

Desventajas:

- Sensible a la indentación.
- Menos eficiente para datos muy grandes.

◇ **5. Parquet y Avro** Formatos binarios orientados al rendimiento, usados en Big Data.

Formato	Características principales	Uso típico
Parquet	Columnares, optimizan lectura y compresión	Data Lakes, Spark, Hadoop
Avro	Orientado a filas, soporta evolución de esquemas	Streaming, Kafka, pipelines ETL

3. 🧩 Logs: Registro de Eventos y Auditoría

Los logs son archivos donde los sistemas registran eventos, errores, solicitudes o métricas.

◇ Propósito

- Monitorear el funcionamiento del sistema.
- Detectar errores o ataques.
- Auditar acciones y cumplimiento normativo.
- Analizar rendimiento o patrones de uso.

◇ Tipos de logs

Tipo	Descripción	Ejemplo
De aplicación	Eventos del software (errores, inicios de sesión).	Error 500: conexión rechazada
De sistema	Actividades del SO (arranques, servicios).	kernel: eth0 up
De acceso	Peticiones a un servicio web.	GET /index.html 200 OK
De seguridad	Autenticación, permisos, auditoría.	User root login failed

◇ Estructura típica de un log Generalmente, cada línea contiene:

[timestamp] [nivel] [origen] [mensaje]

Ejemplo:

```
2025-11-12 10:15:23 INFO Server: Iniciando servicio HTTP
2025-11-12 10:15:30 ERROR DB: Conexión rechazada
```

Campos comunes:

- Timestamp: fecha y hora del evento.
- Nivel: severidad (INFO, WARN, ERROR, DEBUG).
- Origen: componente que genera el mensaje.
- Mensaje: descripción del evento.

4. 🧠 Niveles de severidad de logs

Nivel	Descripción	Ejemplo
DEBUG	Información técnica detallada.	Conectando al host 127.0.0.1
INFO	Eventos normales.	Servicio iniciado
WARN	Posibles problemas.	Espacio en disco bajo
ERROR	Fallo en una operación.	Error al conectar a la base de datos
CRITICAL/FATAL	Fallo grave que detiene el sistema.	Kernel panic

5. 🔍 Centralización y Monitoreo de Logs

En sistemas distribuidos, los logs se recopilan y analizan en herramientas centralizadas.

Ejemplos de plataformas

- ELK Stack (Elasticsearch + Logstash + Kibana)
    - Elasticsearch: almacena y permite búsquedas.
    - Logstash: procesa y transforma logs.
    - Kibana: visualiza métricas y dashboards.
- Grafana Loki: alternativa ligera, orientada a métricas y eventos.
- Graylog: administración de logs centralizada con alertas.
- Fluentd / Fluent Bit: para ingesta y agregación de logs.

Ventajas de la centralización

- Trazabilidad completa.
- Alertas automáticas.
- Detección temprana de fallos.

- Auditoría y cumplimiento de normativas.

6. 🔐 Buenas prácticas en la gestión de logs

- Incluir timestamps normalizados (UTC).
- Registrar niveles adecuados (INFO, ERROR, etc.).
- Evitar datos sensibles (contraseñas, tokens).
- Usar formatos estructurados (JSON log).
- Rotar y comprimir logs antiguos.
- Automatizar alertas y monitoreo.

7. 🧩 Ejemplo de Log en Formato JSON

```
{
  "timestamp": "2025-11-12T10:15:23Z",
  "level": "INFO",
  "service": "auth-api",
  "message": "Inicio de sesión exitoso",
  "user_id": 2041,
  "ip": "192.168.0.25"
}
```

Ventajas:

- Fácil de parsear y filtrar.
- Compatible con herramientas modernas de monitoreo (ELK, Loki, Datadog).

🧠 Conclusión Los formatos de intercambio de datos y los logs estructurados son la base de la comunicación y trazabilidad en entornos modernos. Elegir el formato adecuado depende de:

- La naturaleza de los datos (texto, jerárquico o masivo).
- Los requisitos de interoperabilidad y rendimiento.

Los logs, por su parte, son esenciales para el mantenimiento, seguridad y auditoría de los sistemas distribuidos. Su correcta estructuración y monitoreo permiten anticipar errores y mejorar la eficiencia operativa.

🧩 Introducción a las Expresiones Regulares (REGEX)

📓 ¿Qué son las Expresiones Regulares?

Una **Expresión Regular (REGEX o RegExp)** es una secuencia de caracteres que define un **patrón de búsqueda**.

Podés pensarlas como una versión avanzada de **Ctrl+F**, capaz de buscar coincidencias complejas en textos o datos.

🔍 Permiten buscar

- Números de teléfono en distintos formatos.
- Direcciones de correo electrónico válidas.
- Líneas que comienzan o terminan con ciertas palabras.
- Palabras que siguen un patrón específico (ej. que empiecen con “a” y terminen en “o”).

⚙️ ¿Por qué son útiles?

1. **Validación:** verificar si un texto cumple un formato (por ejemplo, un email o una contraseña).
2. **Búsqueda y extracción:** localizar fragmentos que coincidan con un patrón (por ejemplo, todos los enlaces de una web).
3. **Sustitución:** reemplazar coincidencias por otro texto (por ejemplo, ocultar números de DNI).

🧠 Breve Historia

- Surgen en los **años 50** en el campo de la informática teórica.
- Popularizadas por **Unix**, a través de herramientas como **ed** y **grep** (*globally search for a regular expression and print*), creadas por **Ken Thompson**.
- Hoy son estándar en la mayoría de los lenguajes y editores de texto.

🔡 Conceptos Fundamentales

◇ Caracteres Literales

Coinciden exactamente con el texto.
Ejemplo: el patrón `gato` busca “gato” literal.

Texto	Coincide
El gato negro.	☑
Ágata	❌
El ratón	❌

◇ Metacaracteres: los caracteres con “superpoderes”

Símbolo	Nombre	Función
`.`	Punto	Coincide con cualquier carácter
`\`	Escape	Anula el significado especial
`|`	Alternancia	Operador “OR”

Símbolo	Nombre	Función
`*`	Asterisco	Cero o más repeticiones
`+`	Más	Una o más repeticiones
`?`	Interrogación	Cero o una vez
`{}`	Llaves	Número específico de repeticiones
`[]`	Corchetes	Clase de caracteres
`()`	Paréntesis	Agrupa o captura
`^`	Acento circunflejo	Inicio de línea
`$`	Dólar	Fin de línea

🧭 Anclas y Delimitadores de Palabra

Anclas

- `^` → inicio de línea.
- `$` → fin de línea.

Ejemplos:

- `^Hola` → solo si “Hola” está al inicio.
- `fin$` → solo si “fin” está al final.
- `^Solo esto$` → coincide solo con esa línea exacta.

Delimitador de palabra: `\b`

Indica un límite de palabra.

Ejemplo: `\bgato\b`

Texto	Coincide
El gato negro.	☑
Los gatos.	❌
Estoy agatopado.	❌

🔠 Clases de Caracteres y Rangos

Clases básicas

- `[abc]` → coincide con “a”, “b” o “c”.
- `[^abc]` → coincide con cualquier carácter que **no** sea “a”, “b” o “c”.

Rangos

- `[0-9]` → cualquier dígito.
- `[a-z]` → cualquier letra minúscula.
- `[A-Z]` → cualquier letra mayúscula.
- `[A-Za-z0-9_]` → cualquier carácter alfanumérico o guión bajo.

Clases predefinidas

Atajo	Equivalente	Significado
`\d`	[0-9]	Dígito
`\D`	[^0-9]	No dígito
`\w`	[A-Za-z0-9_]	Carácter de palabra
`\W`	[^A-Za-z0-9_]	No carácter de palabra
`\s`	[\t\r\n\f\v]	Espacio en blanco
`\S`	[^ \t\r\n\f\v]	No espacio

Comodín universal: `.`

Coincide con cualquier carácter (excepto saltos de línea).

Ejemplo: `h.la` → coincide con “hola”, “hila”, “h&la”.

➕ Cuantificadores: “¿Cuántas veces?”

Símbolo	Significado	Ejemplo
`*`	0 o más veces	`ho*la` → “hla”, “hooola”
`+`	1 o más veces	`ho+la` → “hola”, “hooooola”
`?`	0 o 1 vez	`colou?r` → “color” o “colour”
`{n}`	n veces exactas	`\d{4}` → 4 dígitos
`{n,}`	n o más veces	`\d{2,}` → al menos 2 dígitos
`{n,m}`	entre n y m veces	`\w{3,5}` → 3 a 5 caracteres

🧩 Agrupación, Captura y Alternancia

Agrupación `()`

Permite tratar una secuencia como unidad.

Ejemplo: `(ha)+`
Coincide con “ha”, “haha”, “hahaha”.

Captura

Ejemplo: `(\d{2})-(\d{2})-(\d{4})`
Coincide con fechas tipo “25-12-2025” y guarda cada parte (día, mes, año).

Alternancia |

Permite opciones.

Ejemplo: `gato|perro` → busca “gato” o “perro”.
Ejemplo mejorado: `Me gusta el (café|té)` → coincide con ambas frases posibles.

💻 Uso Práctico de REGEX

📝 En editores de texto (VS Code)

1. Presioná `Ctrl+F`.
2. Activá el ícono `.*` (modo REGEX).
3. Escribí el patrón.

Ejemplo: `color:.*(#|rgb)`
Busca todas las líneas que definen colores CSS.

🧠 En JavaScript

```
const patron = /\d{3}-\d{3}-\d{4}/;
const texto = "Tel: 123-456-7890";
console.log(patron.test(texto)); // true
```

Funciones comunes:

- `.test()` → devuelve true/false.
- `.match()` → devuelve coincidencias.
- `.replace()` → reemplaza texto.

🐍 En Python

```
import re
texto = "Precio $19.99"
patron = r"\d+\.\d{2}"
print(re.search(patron, texto).group())  # 19.99
```

Funciones:

- `re.search()` → primera coincidencia.

- `re.findall()` → todas las coincidencias.
- `re.sub()` → reemplazar coincidencias.

🌐 En herramientas online

regex101.com y **regexr.com** permiten:

- Probar patrones y ver coincidencias en tiempo real.
- Explicar cada parte del patrón.
- Usar diferentes "sabores" (Python, JavaScript, etc.).

Son ideales para **aprender y depurar** expresiones regulares.

🧩 Conclusión

Las **expresiones regulares** son una herramienta esencial para el análisis, validación y manipulación de texto. Su dominio mejora la productividad en **programación, ciberseguridad y análisis de datos**, permitiendo búsquedas avanzadas y automatización precisa.

🗄️ Módulo 8: Bases de Datos No Relacionales (NoSQL)

📋 Contenido General

1. Origen de NoSQL
2. Limitaciones del modelo relacional
3. Nuevo paradigma NoSQL
4. Familias NoSQL
5. Teorema CAP
6. Data Warehousing y OLAP
7. Resumen y decisiones

---

1. 🌐 Origen de NoSQL

◇ De la web estática a la era distribuida

En los años 90, la web era **estática y con pocos usuarios**, por lo que los modelos relacionales eran suficientes.
Sin embargo, entre **2004 y 2008**, con el auge de **Google, Facebook, Amazon y Twitter**, surgió la necesidad de manejar **millones de usuarios y datos diversos** (logs, fotos, sensores, geolocalizaciones).

◇ Cambio de paradigma

El crecimiento exponencial reveló las **limitaciones de las bases relacionales**, impulsando la búsqueda de soluciones más flexibles y escalables.

◇ Nacimiento de NoSQL

NoSQL emergió para ofrecer:

- **Escalabilidad horizontal**
- **Flexibilidad de esquemas**
- **Alta disponibilidad global**

Influencias y eventos clave

- **2007 – Paper Dynamo (Amazon):** demostró que era posible escalar sin sacrificar disponibilidad.
- **2009 – Lanzamiento de MongoDB y meetup NoSQL:** el término "NoSQL" se acuñó para describir bases que rompían con el modelo relacional tradicional.

---

2. Limitaciones del Modelo Relacional

◇ Escalabilidad vertical insostenible

1. **Costo:** crecer requiere servidores más potentes (CPU, RAM), lo cual **no escala económicamente**.
2. **Límites físicos:** llega un punto donde **no se puede seguir ampliando** el hardware.
3. **Impacto en el negocio:** no se adapta al crecimiento rápido de usuarios.
4. **Solución:** escalar **horizontalmente** con muchos servidores económicos (NoSQL).

◇ Rigidez de esquemas y migraciones

- Los **esquemas fijos** requieren cambios costosos y bloqueos al modificar tablas.
- Los **datos no estructurados** (JSON, XML, IoT) no encajan bien en modelos tabulares.
- Las migraciones lentas obstaculizan el **desarrollo ágil**.

---

3. Nuevo Paradigma NoSQL

◇ Diseñadas para la web moderna

- **Escalabilidad horizontal:** se agregan nodos económicos en lugar de ampliar servidores caros.
- **Esquemas flexibles:** permiten campos dinámicos sin detener la aplicación.
- **Alta disponibilidad y baja latencia:** respuestas globales en milisegundos, con **consistencia eventual**.

◇ Comparativa: SQL vs NoSQL

Característica	SQL	NoSQL
Esquemas	Fijos y normalizados	Flexibles y dinámicos
Escalabilidad	Vertical	Horizontal (sharding, replicación)
Transacciones	ACID (seguras, consistentes)	BASE (disponibles, consistencia eventual)
Lenguaje	SQL estándar	APIs específicas (CQL, JSON, Cypher, etc.)

---

4. 🗃️ Familias NoSQL

◇ 1. Clave-Valor

- **Ejemplos:** Redis, DynamoDB
- **Características:** Pares `clave → valor` con acceso O(1).
- **Ventajas:** latencia submilisegundo, gran velocidad y simplicidad.
- **Usos:** cachés, sesiones, contadores y colas ligeras.

```
SET user:1234:cart '[{"sku":"A1","qty":2}]'
GET user:1234:cart
INCR post:987:likes
```

◇ 2. Documental (JSON/BSON)

- **Ejemplos:** MongoDB, Couchbase
- **Características:** documentos JSON sin esquema fijo.
- **Ventajas:** consultas ricas sin JOINs, indexación flexible.
- **Usos:** perfiles de usuario, catálogos y CMS.

◇ 3. Columnar

- **Ejemplos:** Cassandra, HBase
- **Características:** datos organizados en columnas agrupadas en familias.
- **Ventajas:** optimiza escritura masiva y compresión.
- **Usos:** series temporales, IoT, analytics, métricas globales.

◇ 4. Grafo

- **Ejemplos:** Neo4j, Amazon Neptune
- **Características:** nodos y relaciones con propiedades.
- **Ventajas:** consultas de relaciones complejas en tiempo constante.
- **Usos:** redes sociales, detección de fraude, sistemas de recomendación.

> Las consultas de grafos permiten analizar *caminos, centralidad y patrones de interacción* sin JOINs costosos.

---

5. ⚖️ Teorema CAP

◇ Definición

En sistemas distribuidos **solo pueden cumplirse dos de tres propiedades**:

- **C:** Consistencia → todos ven el mismo dato al mismo tiempo.
- **A:** Disponibilidad → siempre responde, aunque con datos viejos.

- **P:** Tolerancia a particiones → sigue funcionando aunque haya cortes de red.

◇ Tipos según CAP

Tipo	Propiedades	Ejemplos	Prioriza	Sacrifica	Analogía
CA	Consistencia + Disponibilidad	MySQL local	Datos exactos	Tolerancia a fallas	Funciona solo si la red no falla
CP	Consistencia + Particiones	MongoDB (modo fuerte), HBase	Precisión	Disponibilidad	Cajero que no opera si pierde conexión
AP	Disponibilidad + Particiones	Cassandra, DynamoDB	Responde siempre	Consistencia temporal	Tienda que sigue vendiendo sin conexión

◇ ACID vs BASE

Modelo	Propósito	Uso típico
ACID	Transacciones seguras y coherentes	Banca, contabilidad
BASE	Alta disponibilidad con consistencia eventual	Redes sociales, comercio electrónico

---

6. 🏢 Data Warehousing y OLAP

◇ Del OLTP al Data Warehouse

- **Data Warehouse (DW):** centraliza datos históricos por temas de negocio.
- Usa **ETL** y modelos **estrella o copo de nieve**.
- **OLAP:** permite análisis *slice-dice*, *drill-down* y agregaciones precalculadas.

◇ Beneficios

- Integración de múltiples fuentes.
- Análisis histórico y estratégico.
- Consultas complejas sin afectar operaciones diarias.

◇ Uso moderno

- Reportes gerenciales y dashboards.
- Análisis de tendencias y segmentación de clientes.
- Aunque los **Data Lakes** crecen, el **DW sigue siendo esencial** para reportes curados y cumplimiento normativo.

---

7. 📊 Resumen y Decisiones

◇ Guía rápida de elección

Tipo	Ideal para	Características
Clave-valor	Caché, sesiones	Velocidad y simplicidad
Documental	Contenidos dinámicos	Estructuras flexibles
Columnar	Series temporales, métricas	Alta ingesta y consultas por rango
Grafo	Redes sociales, fraude	Análisis de relaciones

🧠 Conclusión

- **NoSQL** nació para enfrentar los límites de las bases relacionales en la era web global.
- Permite **escalar horizontalmente**, usar **esquemas flexibles** y mantener **alta disponibilidad**.
- El **teorema CAP** recuerda que no hay solución perfecta: se deben equilibrar consistencia, disponibilidad y tolerancia a fallos.
- La **arquitectura ideal** combina ambos mundos (SQL y NoSQL), según el contexto y los requerimientos del sistema.

🧠 Módulo 7: Otras Tendencias y Tecnologías Actuales

📄 Contenido General

1. ETL Moderno
2. Big Data
3. Ciencia de Datos
4. Tendencias 2025

---

1. ⚙️ ETL Moderno

◇ Del dato bruto al insight

El proceso ETL (Extract, Transform, Load) evoluciona hacia un enfoque **híbrido ETL/ELT**, integrando **Data Lakes y soluciones en la nube**.
Este modelo garantiza:

- **Calidad y trazabilidad** de los datos antes del análisis.
- Integración de **fuentes modernas** como APIs, IoT y redes sociales.

◇ Importancia de la gobernanza

La **gobernanza de datos** es esencial para asegurar la integridad, seguridad y consistencia de la información. Sin ella, los análisis pueden resultar **frágiles o inexactos**.

◇ Etapas del proceso ETL

1. Extracción

- Fuentes: bases **SQL**, **NoSQL**, **Kafka streaming** y archivos **Parquet**.
- **Desafíos técnicos:** necesidad de conectores dinámicos, autenticación OAuth y gestión de *rate limits*.

2. Transformación

- **Limpieza de datos:** eliminación de duplicados, homologación de unidades y monedas.
- **Enriquecimiento:** agregación de datos adicionales (ej. geolocalización).
- **Validación:** verificación de integridad referencial.
- **Versionado:** reglas de negocio en **Git**, ejecutadas con **Spark** o **SQL**.

3. Carga

- **Tipos:** completa, incremental o *streaming*.
- **Destinos:** Data Warehouses, Data Lakes o nubes (ej. Snowflake).
- **Objetivo:** optimizar costo y rendimiento mediante formato y partición adecuados.

---

2. 💾 Big Data

◇ Las 5V del Big Data

1. **Volumen**: grandes cantidades de datos.
2. **Velocidad**: generación y actualización rápida.
3. **Variedad**: datos estructurados, semiestructurados y no estructurados.
4. **Veracidad**: precisión y confiabilidad.
5. **Valor**: información útil para la toma de decisiones.

◇ Arquitectura distribuida típica

- **Ingesta:** Kafka, NiFi.
- **Almacenamiento:** HDFS, S3.
- **Procesamiento:** Spark, Flink.
- **Consulta:** Presto, Hive.
- **Visualización:** Kibana, Superset.

Orquestación y seguridad: Airflow gestiona flujos de datos, con políticas de réplica y control de acceso.
Modularidad: permite escalabilidad y adaptación a las necesidades del negocio.

◇ MapReduce vs Spark

Característica	MapReduce	Spark
Rendimiento	Más lento (lectura/escritura en disco)	Hasta 100x más rápido (procesa en memoria)
Uso	Batch	Batch y tiempo real
Ventajas	Tolerancia a fallos, económico	Compatible con múltiples lenguajes, escalable, soporta ML y SQL

Característica	MapReduce	Spark
Ideal para	Procesos masivos y económicos	Cargas mixtas y análisis en tiempo real

◇ Bases de datos distribuidas clave

- **MongoDB**: NoSQL documental, escalabilidad horizontal.
- **Elasticsearch**: ideal para búsquedas textuales y logs.
- **Cassandra**: alta disponibilidad y escalabilidad.
- **HBase**: consistencia fuerte sobre HDFS.

---

3. 📊 Ciencia de Datos

◇ Ciclo de vida completo

1. **Recolección de datos:** desde SQL, APIs e IoT.
2. **Análisis exploratorio (EDA):** identificación de patrones con matplotlib y seaborn.
3. **Limpieza y preparación:** uso de pandas, Airflow.
4. **Modelado y despliegue:** scikit-learn, TensorFlow y evaluación de métricas.

◇ Rol de los sistemas de datos

- **Fuentes confiables:** bases transaccionales, IoT, APIs.
- **Integración:** ETL/ELT y almacenamiento seguro.
- **Procesamiento:** batch o streaming.
- **Exposición analítica:** dashboards e insights.

◇ Perfiles profesionales

Rol	Función
Data Engineer	Diseña y mantiene pipelines de datos.
Data Analyst	Interpreta y visualiza datos.
Data Scientist	Crea modelos predictivos y analíticos.
ML Engineer	Despliega modelos en producción.

La **colaboración** entre estos roles garantiza proyectos exitosos.

---

4. 🚀 Tendencias 2025

◇ Lakehouses y formatos abiertos

- **Ejemplos:** Delta Lake, Iceberg, Hudi.

- **Ventajas:**
  - Transacciones **ACID** y control de versiones.
  - Interoperabilidad y reducción de dependencias propietarias.
  - Menor duplicación y optimización de costos.

◇ Streaming en tiempo real

- **Herramientas:** Kafka Streams, Flink, Spark Structured Streaming.
- **Aplicaciones:**
  - Ciberseguridad (alertas).
  - Recomendaciones personalizadas.
  - Dashboards con latencia subsegundo.

◇ IA generativa y embeddings

- **Usos:** generación de resúmenes, clasificación de tickets, búsqueda semántica.
- **Integración:** FastAPI y ONNX sin reescribir pipelines ETL.
- **Aplicaciones:** insights automáticos y optimización de procesos.

◇ Privacidad y gobernanza

- **Normativas:** GDPR, CCPA.
- **Prácticas:**
  - Políticas de acceso dinámico.
  - Auditorías continuas.
  - Herramientas open-source para transparencia.

◇ Edge y Federated Learning

- **Procesamiento local:** reduce tráfico y preserva privacidad.
- **Entrenamiento distribuido:** solo se comparten gradientes.
- **Usos:** defensa, salud y entornos sensibles.

◇ Sostenibilidad energética

- Clusters con **GPUs eficientes**.
- **Compresión de datos** y **scheduling dinámico** para reducir huella de carbono.

🔮 Hacia 2026 y Más Allá

🌐 Modelos multimodales

Combinan texto, imágenes y datos estructurados.
Impulsan nuevas capacidades en la Ciencia de Datos.

Convergencia tecnológica

Integración de **lakehouse-serverless** y **tecnologías cuánticas** para optimización.

Regulaciones globales

Nuevas normas de privacidad y seguridad influirán en la arquitectura de datos.

Preparación para el futuro

- Arquitecturas **modulares**.
- Cultura **data-driven**.
- **Formación continua** en ciberseguridad.

Conclusión

El ecosistema de datos avanza hacia la **automatización, escalabilidad y sostenibilidad**.
Las organizaciones que adopten modelos **híbridos, éticos y eficientes** estarán preparadas para el futuro digital.